Symulacja drapieżnik–ofiara z uczeniem ze wzmocnieniem (implementacja wg Park et al., 2021)

—

9 kwietnia 2026

Streszczenie

[Streszczenie całej pracy — do uzupełnienia po napisaniu wstępu i części porównawczej.]

1 Inne podejścia do problemu

[Tu: opis wcześniejszych lub alternatywnych podejść do modelowania dynamiki drapieżnik–ofiara — literatura, prostsze modele, inne metody RL itp. Fragment do uzupełnienia.]

Szkic podsekcji (opcjonalnie)

[Do uzupełnienia.]

2 Implementacja według Park et al. (2021)

Niniejsza część opisuje środowisko, uczenie i wyniki zgodne z pracą Park et al. [**?**], w powiązaniu z modułami `park_env` oraz `park_train` w repozytorium projektu.

2.1 Środowisko symulacyjne

Świat jest dyskretną siatką $N \times N$ komórek (domyślnie $N = 50$). W każdej komórce może znajdować się co najwyżej jeden agent (drapieżnik lub ofiara). Granice są *okresowe* (torus): ruch i obserwacja lokalna zawijają się modulo N, analogicznie do założeń w [**?**]. Agenci nie komunikują się jawnie.

Inicjalizacja epizodu. Losowa permutacja komórek; następnie rozmieszczenie $n_X = 100$ drapieżników i $n_Y = 500$ ofiar na pustych polach. Wartości domyślne: klasa `ParkEnvConfig` w pliku `park_env/constants.py`.

Dynamika drapieżnika. W jednym kroku środowiska drapieżnik wybiera jedną z dziewięciu akcji (osiem sąsiedztw Moore'a lub pozostanie). Jeśli pole docelowe jest puste — przechodzi tam. Jeśli zajmuje je inny drapieżnik — pozostaje na miejscu. Jeśli zajmuje je ofiara — drapieżnik ją eliminuje: otrzymuje nagrodę $+1$, ofiara -1 (formalny zapis nagród w podsekcji **??**). Po udanym polowaniu z prawdopodobieństwem $b_X = 0{,}2$ (rozkład Bernoulliego) w *komórce startowej* pojawia się potomek drapieżnika. Każdy drapieżnik ma licznik głodu; jeśli w danym kroku nie zjadł ofiary, głód rośnie o 1. Przy głodzie $\geq T_X = 15$ agent jest usuwany.

Dynamika ofiary. Ofiara również wybiera dziewięć akcji. Przejście na puste pole: ruch; z prawdopodobieństwem $b_Y = 0{,}6$ w komórce opuszczonej pojawia się potomek. Jeśli pole docelowe zajmuje inna ofiara — brak ruchu. Jeśli drapieżnik — ofiara jest schwytana (nagroda jak wyżej). Wiek ofiary rośnie co krok; przy wieku $\geq T_Y = 30$ agent jest usuwany.

Szczegół implementacyjny. W kodzie (`park_env/grid_env.py`) akcje wszystkich agentów w danym kroku są rozstrzygane *sekwencyjnie* w ustalonej kolejności (y, x, id), co wpływa na szczegółową mikrodynamikę względem równoległego modelu ruchu.

2.2 Obserwacje, przestrzeń akcji i zakończenie epizodu

Zgodnie z [**?**] każdy agent widzi lokalne okno $r \times r$ komórek (domyślnie $r = 5$). W implementacji wejście ma postać tensora $5 \times 5 \times 3$ spłaszczonego do wektora: trzy kanały one-hot oznaczają komórkę pustą, drapieżnika lub ofiarę (`local_window_one_hot` w `park_env/observation.py`), z zawijaniem jak na torusie.

Przestrzeń akcji ma 9 elementów (numeracja $0, \ldots, 8$ jak na Rys. 2 w [**?**]; mapowanie na przesunięcia (dx, dy) w `ACTION_DELTAS`).

Epizod kończy się (*terminated*), gdy liczba drapieżników lub ofiar spadnie do zera. Dodatkowo stosowany jest limit `max_episode_steps` $= 10\,000$ (*truncated*) — standardowe ograniczenie numeryczne w RL.

Nagrody są antagonistyczne, jak w [**?**]:

$$R_{\text{drapieżnik}} = \begin{cases} +1 & \text{jeśli w wyniku akcji schwytał ofiarę,} \\ 0 & \text{w przeciwnym razie,} \end{cases} \tag{1}$$

$$R_{\text{ofiara}} = \begin{cases} -1 & \text{jeśli w wyniku akcji została schwytana,} \\ 0 & \text{w przeciwnym razie.} \end{cases} \tag{2}$$

2.3 Sieci neuronowe i polityki

Stosowane jest *parameter sharing* z decentralną egzekucją: wszystkie drapieżniki współdzielą jedną sieć polityki i jeden proces uczenia, a wszystkie ofiary — drugi (`pred_algae`, `prey_algae` w `CoevolutionTrainer`). Warunkowa-

nie na lokalnej obserwacji s_i zachowuje heterogeniczność zachowań mimo wspólnych wag.

Aktor to wielowarstwowy perceptron (MLP): dwie warstwy ukryte po 256 neuronach z aktywacją ReLU, wyjście — logity 9 akcji i rozkład kategoryczny (`DiscreteActor` w `park_train/policies.py`). Krytyk to podwójne $Q(s, a)$: wejście stanu i akcji w postaci one-hot (`DoubleQCritic`).

2.4 Uczenie: AlgaeDICE i pętla koewolucji

Uczenie odbywa się w duchu procedury z Tabeli 1 w [**?**]: naprzemienne fazy przybliżonych *best response* dla przeciwnika w postaci ustalonej polityki oraz krótsza faza wspólnych aktualizacji. W repozytorium jedna zewnętrzna iteracja (`CoevolutionTrainer.iteration`) zbiera doświadczenia z horyzontem próbkowania $h = 70$ kroków środowiska, następnie wykonuje aktualizacje AlgaeDICE na buforze ofiar (faza BR), potem na buforze drapieżników (BR), a na końcu krótsze aktualizacje dla obu gatunków (*joint*). Domyślnie (`train_park.py`): 4+4 przejść BR i 2+2 joint na iterację, rozmiar minibatcha 256, pojemność bufora ok. $5 \cdot 10^5$ przejść, $\gamma = 0{,}99$.

Algorytm `DiscreteAlgaeDICE` (`park_train/algaedice.py`) implementuje wariant dyskretno-akcyjny z funkcją $f(x) = x^2/2$ w sensie pracy, mieszaniem sieci docelowej krytyka ($\tau = 0{,}005$), optymalizatorem AdamW oraz regularizacją entropii (uczenie temperatury względem celu zbliżonego do entropii rozkładu jednostajnego po akcjach).

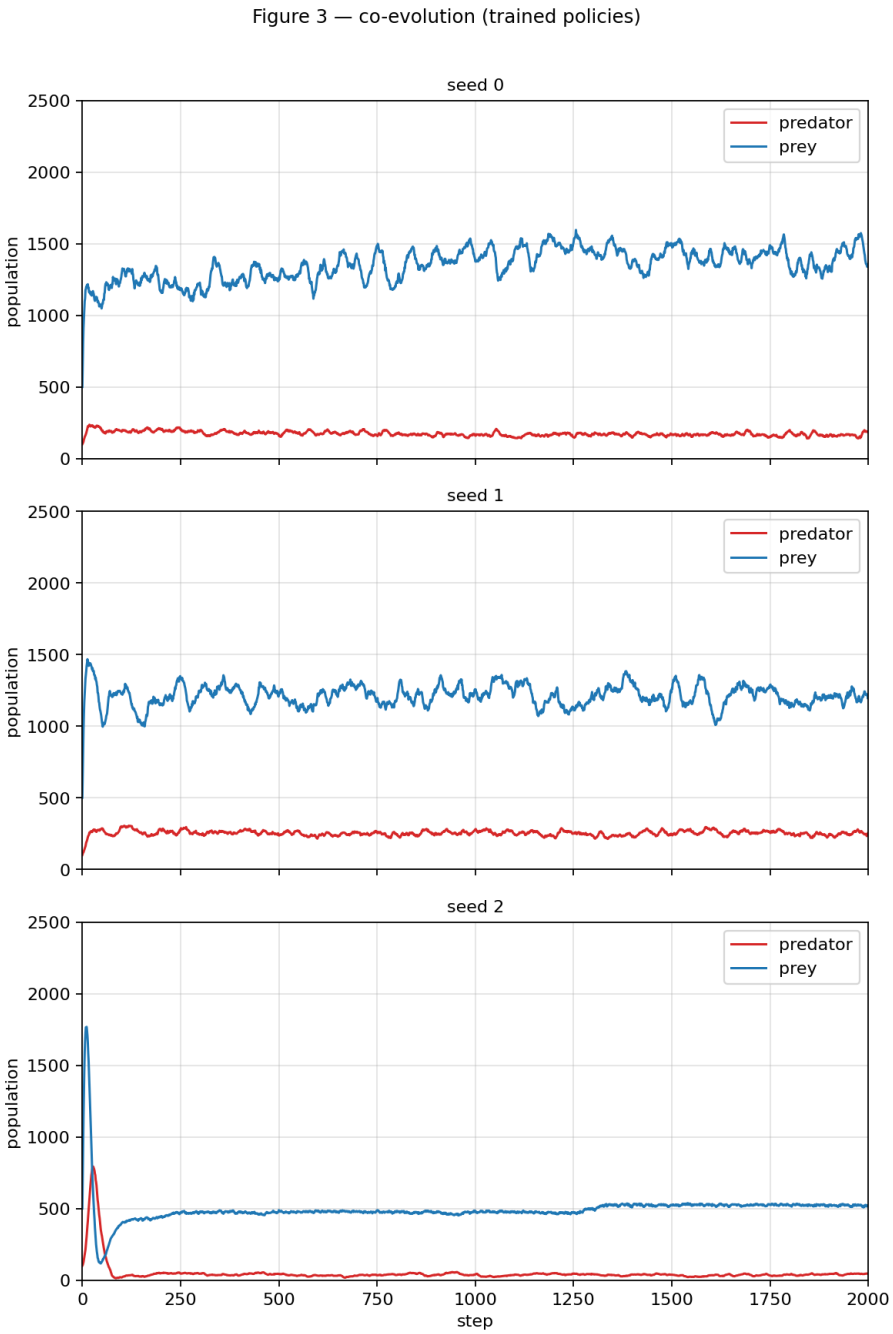
Parametry środowiska i próbkowania zestawiono w tab. **??**.

Tabela 1: Parametry symulacji zgodne z Tabelą 2 w Park et al. [**?**] i wartościami domyślnymi w `ParkEnvConfig` / `train_park.py`.

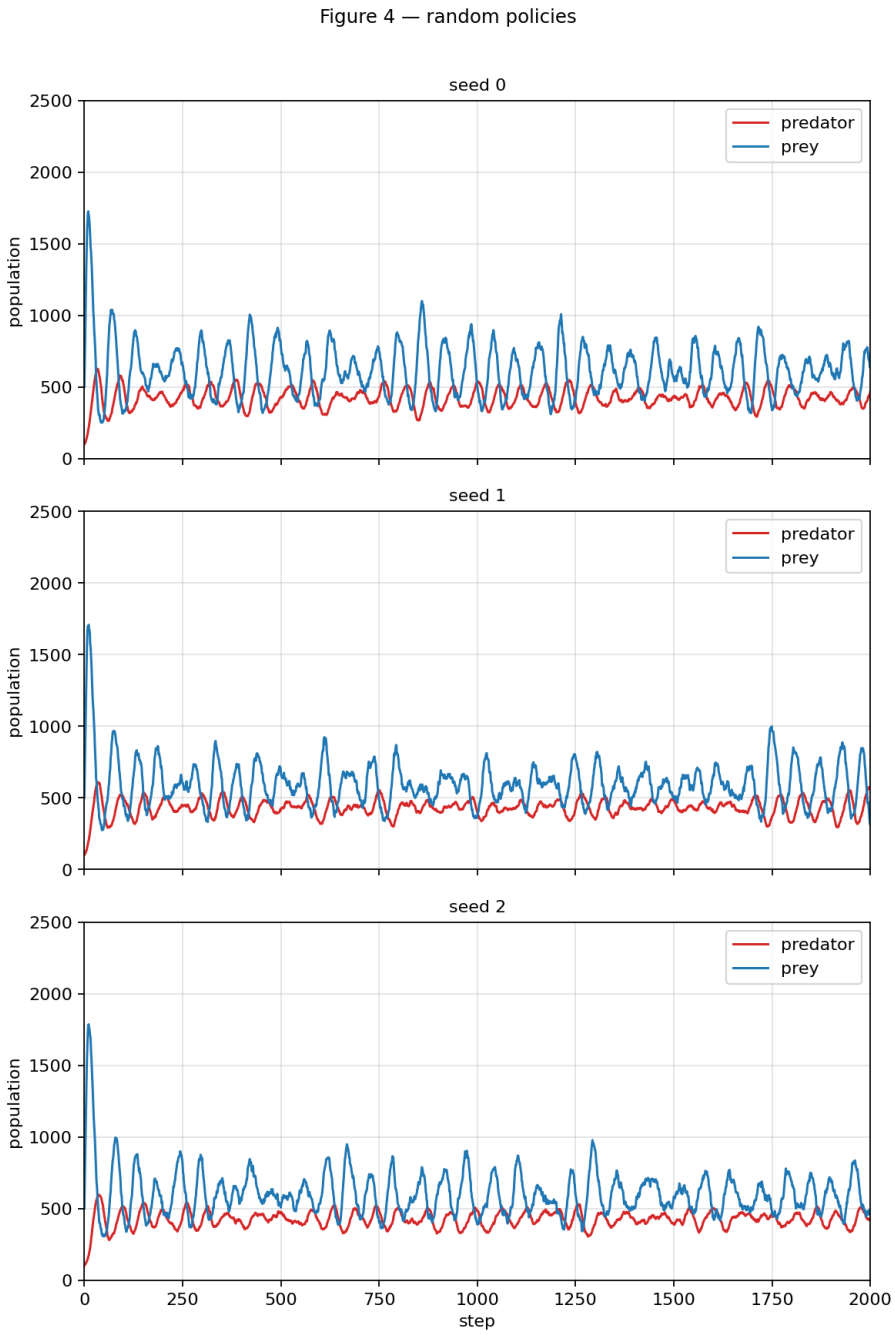
Symbol / wielkość	Wartość
N	50
n_X, n_Y	100, 500
b_X, b_Y	0,2, 0,6
T_X, T_Y	15, 30
r (okno obserwacji)	5
h (horyzont próbkowania)	70
γ	0,99
max. kroków epizodu	10 000

2.5 Wyniki ilustracyjne

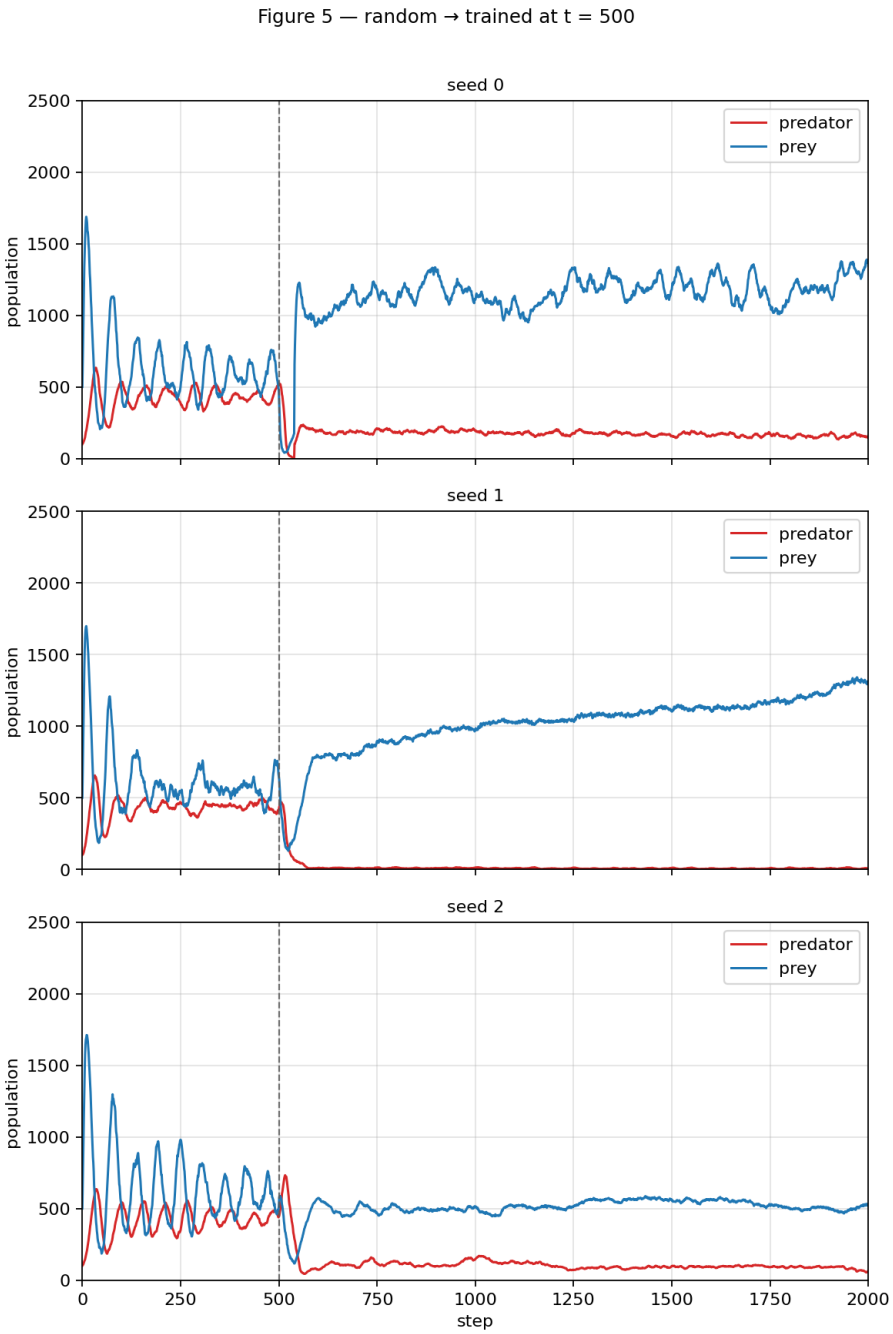
Rysunki **??**–**??** powstały przy użyciu skryptu `reproduce_paper_figures.py`: ślad liczebności drapieżników i ofiar w funkcji kroku środowiska (2000 kroków), dla trzech ziaren losowych, na siatce $N = 50$ i parametrach jak wyżej.



Rysunek 1: Populacja drapieżników i ofiar przy **wytrenowanych** politykach po koewolucji (odpowiednik Fig. 3 w [**?**]). Trzy panele — trzy ziarna losowe.



Rysunek 2: Te same ustawienia osi i ziarna, lecz **jednostajnie losowe** akcje agentów — linia bazowa (Fig. 4 w [**?**]).



Rysunek 3: Pierwsze 500 kroków z losowymi akcjami, następnie polityki po uczeniu; pionowa linia przy $t = 500$ (Fig. 5 w [?]).

Na rys. **??** widać, że po uczeniu przebiegi bywają stabilniejsze lub inaczej rozłożone niż przy polityce losowej (rys. **??**), przy czym konkretny kształt oscylacji zależy od ziarna (np. gorszy reżim dla jednego z seedów). Rysunek **??** ilustruje hipotezę z [**?**]: po przełączeniu na polityki wytrenowane dynamika często stabilizuje się po fazie dużych wahań; jednocześnie pojedyncze ziarna mogą prowadzić do niemal całkowitego wygaszenia drapieżników lub niskiej gęstości — co podkreśla wrażliwość na stochastykę przy tej samej procedurze treningowej.

2.6 Narzędzia eksperymentalne w repozytorium

- `train_park.py` — interfejs CLI do pętli koewolucji; zapis wykresu populacji po iteracjach zewnętrznych (PNG/CSV), opcjonalnie podgląd Pygame.
- `reproduce_paper_figures.py` — trening per ziarno oraz generacja rysunków typu Fig. 3–5 i plików CSV ze śladami kroków.

2.7 Wnioski

- Implementacja odtwarza kluczowe elementy metody Park et al. [**?**]: częściowo obserwowalną grę na siatce z torusem, współdzielone polityki per gatunek, nagrody ((**??**)–(**??**)) oraz koewolucję z AlgaeDICE i horyzontem $h = 70$.
- Porównanie rys. **??** i **??** sugeruje, że uczenie ze wzmocnieniem zmienia jakościowo dynamikę populacji względem losowej polityki, zgodnie z intencją pracy źródłowej.
- Eksperyment z rys. **??** pokazuje behawioralne przejście od eksploracji losowej do polityk uczenia — z różnym skutkiem dla różnych ziaren, co warto traktować jako ograniczenie pojedynczego treningu bez dodatkowej selekcji hiperparametrów.
- Ograniczenia repozytorium względem pełnej pracy [**?**]: brak gotowej reprodukcji wszystkich rysunków jakościowych (np. trajektorie przestrzenne jak Fig. 6–7), deterministyczna kolejność ruchów agentów oraz domyślna liczba zewnętrznych iteracji (30) ustawiona pod wygodę eksperymentu, a nie identycznie z opisem „kilkudziesięciu” iteracji w artykule.

Literatura

[1] J. Park, J. Lee, T. Kim, I. Ahn, J. Park. Co-Evolution of Predator-Prey Ecosystems by Reinforcement Learning Agents. *Entropy* 2021, *23*(4), 461. https://doi.org/10.3390/e23040461